ASSESSMENT REPORT

Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation

DevSecOps Engineering and Application Security

Name	T G JAI KOUSICK
Register Number	192421367
Subject	DevSecOps Engineering and Application Security
Subject Code	CSA5802
Department	B.Tech Information Technology

1. Vulnerability Detection

Vulnerability detection is the systematic process of identifying weaknesses in software, infrastructure, configurations, and dependencies that could be exploited by an attacker. In a DevSecOps context, detection is automated, continuous, and integrated into every stage of the delivery pipeline (shift-left) as well as runtime (shift-right).

1.1 Categories of Vulnerability Detection

Static Application Security Testing (SAST)

Analyses source code, bytecode, or binary without executing the application. Detects coding flaws such as SQL injection, XSS, insecure deserialization, hard-coded secrets, and weak cryptography. Tools: SonarQube, CodeQL, Semgrep, Checkmarx, Fortify. Ideal for early feedback in the IDE and CI pipeline.

Software Composition Analysis (SCA)

Identifies known vulnerabilities in third-party libraries and open-source components by matching against vulnerability databases (NVD, OSV, vendor advisories). Also generates Software Bill of Materials (SBOM). Tools: Snyk, Dependabot, Trivy, OWASP Dependency-Check, Black Duck. Critical for supply-chain security.

Dynamic Application Security Testing (DAST)

Tests a running application by sending crafted requests and observing responses. Finds runtime issues such as authentication bypass, misconfigurations, and injection flaws that static analysis may miss. Tools: OWASP ZAP, Burp Suite, Nuclei, Acunetix. Typically run against staging or ephemeral test environments.

Interactive Application Security Testing (IAST)

Combines elements of SAST and DAST by instrumenting the application at runtime with agents that observe code execution and data flow during functional or security tests. Provides higher accuracy and fewer false positives.

Infrastructure-as-Code (IaC) & Configuration Scanning

Scans Terraform, CloudFormation, Kubernetes manifests, Helm charts, and Dockerfiles for insecure defaults (open security groups, privileged containers, missing resource limits, public S3 buckets). Tools: Checkov, tfsec, Terrascan, KICS, Trivy config.

Container & Image Scanning

Analyses container images for OS package vulnerabilities, language library issues, secrets, and misconfigurations before they are pushed to or pulled from a registry. Tools: Trivy, Clair, Anchore, Aqua, ECR/ACR native scanners.

Secret Detection

Scans code, commit history, configuration files, and CI logs for accidentally committed credentials, API keys, and certificates. Tools: Gitleaks, TruffleHog, detect-secrets, GitHub secret scanning.

Runtime / Continuous Detection

Monitors running workloads for anomalous behaviour, known exploit patterns, and policy violations. Tools: Falco, Sysdig Secure, Aqua Runtime, AWS GuardDuty, cloud CSPM solutions.

1.2 Detection Strategy in DevSecOps Pipelines

- Pre-commit / IDE plugins — immediate developer feedback (SAST lite, secret scanning).
- Pull Request checks — mandatory SAST, SCA, IaC, and secret scans; PR blocked on critical findings.
- CI main-branch pipeline — full SAST, SCA, image build + scan, SBOM generation, signing.
- Pre-deployment / staging — DAST and/or IAST against ephemeral environments.
- Admission control — cluster policies reject unsigned or vulnerable images.
- Runtime continuous monitoring — Falco rules, vulnerability re-scanning of running images, CSPM.

2. Severity Analysis

Severity analysis assigns a risk rating to each detected vulnerability so that remediation can be prioritised. Raw CVSS scores alone are insufficient; context (exploitability, asset criticality, exposure, compensating controls) must be incorporated to produce an actionable severity.

2.1 Common Severity Scoring Systems

CVSS (Common Vulnerability Scoring System) — Industry-standard quantitative score (0.0–10.0) with Base, Temporal, and Environmental metrics. Base score reflects intrinsic characteristics; Environmental metrics allow organisations to adjust for their specific deployment context.

EPSS (Exploit Prediction Scoring System) — Probability (0–1) that a vulnerability will be exploited in the wild in the next 30 days. Complements CVSS by adding real-world threat likelihood.

SSVC (Stakeholder-Specific Vulnerability Categorization) — Decision-tree framework from CISA that produces action-oriented outcomes (Track, Track*, Attend, Act) based on exploitation status, technical impact, and mission/business criticality.

2.2 Typical Severity Bands Used in Pipelines

Severity	CVSS Range (typical)	Pipeline Action	Remediation SLA (example)
Critical	9.0 – 10.0	Block merge / deployment	24–48 hours
High	7.0 – 8.9	Block or require exception	7 days
Medium	4.0 – 6.9	Warn; allow with ticket	30 days
Low	0.1 – 3.9	Informational / backlog	90 days or next release
Info / None	0.0	Log only	As capacity allows

Table 1: Severity Bands and Example Pipeline Actions / SLAs

2.3 Contextual Risk Factors Beyond CVSS

- Asset criticality — Is the component Internet-facing? Does it handle PII, payment data, or authentication?
- Exploitability — Is a public exploit available? Is the vulnerability being actively exploited (KEV catalog)?
- Exposure — Network reachability, authentication requirements, attack complexity.
- Compensating controls — WAF rules, network segmentation, runtime protection that reduce residual risk.
- Business impact — Potential for data breach, service outage, regulatory fine, or reputational damage.
- Fix availability — Is a patch or upgraded dependency available without breaking changes?

A practical approach used by many organisations is: **Risk Score = f(CVSS Base, EPSS, Asset Criticality, Exposure)**. Only vulnerabilities exceeding a defined risk threshold (or appearing on the CISA KEV list) automatically fail the pipeline.

3. Security Verification Workflow Simulation

A security verification workflow defines the ordered set of automated and human gates that a change must pass before it is considered safe to deploy. Simulation allows teams to model, test, and optimise this workflow without risking production systems.

3.1 Reference Security Verification Workflow

```
[ Developer Commit / PR ]
        |
        v
[ 1. Secret Scan + Pre-commit hooks ] ---- FAIL --> Block + Notify
        |
       PASS
        v
[ 2. SAST + SCA + IaC Scan ] ------------ FAIL --> Block / Create Ticket
        |
       PASS
        v
[ 3. Unit + Integration Tests ]
        |
        v
[ 4. Build Image + Image Scan + SBOM + Sign ]
        |
       PASS (signed artifact only)
        v
[ 5. Update GitOps Desired State (PR) ]
        |
        v
[ 6. Policy-as-Code Validation (OPA/Kyverno) ]
        |
       PASS
        v
[ 7. Deploy to Staging / Ephemeral Env ]
        |
        v
[ 8. DAST / IAST / Smoke Security Tests ] -- FAIL --> Rollback + Ticket
        |
       PASS
```

```
        v
[ 9. Manual / CAB Approval (if required for Prod) ]
        |
        v
[ 10. Progressive Delivery to Production ]
         (Canary / Blue-Green + automated analysis)
        |
        v
[ 11. Runtime Monitoring + Continuous Re-scan ]
```

Figure 1: End-to-End Security Verification Workflow

3.2 Simulation Approach

- **Model the pipeline as a state machine or directed graph** — each security gate is a node with pass/fail transitions and timing attributes.
- **Inject synthetic findings** — use known vulnerable packages (e.g., intentionally outdated libraries), deliberately insecure IaC, or test secrets to exercise fail paths.
- **Measure gate effectiveness** — true-positive rate, false-positive rate, mean time to detect (MTTD), and mean time to remediate (MTTR) for each stage.
- **Stress-test exception handling** — verify that time-bound exceptions, risk-acceptance workflows, and break-glass procedures behave as designed and are fully audited.
- **Validate progressive delivery safety nets** — confirm that canary analysis correctly aborts on security regression or elevated error rates.
- **Record metrics for every simulated run** — pipeline duration, number of blocked builds, severity distribution of findings, and developer feedback loop time.

3.3 Example Simulation Scenarios

- Introduce a library with a Critical CVSS + high EPSS score → expect CI to fail at SCA stage and create a blocking ticket.
- Commit a hard-coded AWS key → secret scanner must fail the PR within seconds.
- Deploy a container running as root with a privileged securityContext → admission controller (Kyverno/OPA) must reject the pod.
- Promote an image that was signed but later found to contain a newly disclosed Critical vulnerability → runtime re-scan or continuous SCA should alert and optionally trigger rollback.
- Simulate a canary deployment that increases 5xx rate or triggers Falco alerts → progressive delivery controller must halt promotion and roll back.

4. Performance Evaluation

Performance evaluation measures both the **security effectiveness** of the detection and verification system and the **impact on delivery velocity**. A secure pipeline that is too slow or generates excessive false positives will be bypassed; therefore both dimensions must be quantified and continuously improved.

4.1 Key Performance Indicators (KPIs)

Category	Metric	Description / Target (example)
Detection Effectiveness	True Positive Rate (TPR)	Share of real vulnerabilities correctly identified; target > 90% for Critical/High
Detection Effectiveness	False Positive Rate (FPR)	Share of findings that are not exploitable; target < 15% after tuning
Detection Effectiveness	Coverage	% of code / dependencies / images scanned; target 100% of production artifacts
Speed / Developer Experience	Mean Pipeline Duration	End-to-end CI time including security gates; target < 15–20 min for typical service
Speed / Developer Experience	Time-to-Feedback	Minutes from commit to first security signal; target < 5 min for PR checks
Risk Reduction	MTTD (Mean Time to Detect)	Time from vulnerability introduction to detection; target hours for Critical
Risk Reduction	MTTR (Mean Time to Remediate)	Time from detection to fix merged; aligned with severity SLAs
Risk Reduction	Escape Rate	% of vulnerabilities that reach production; target near zero for Critical/High
Process Health	Exception / Waiver Rate	% of builds that proceed via exception; should trend downward
Process Health	Policy Compliance Score	% of deployments satisfying all admission and runtime policies

Table 2: Core KPIs for Security Verification Performance Evaluation

4.2 Evaluation Methodology

- **Baseline measurement** — Record current pipeline duration, finding volumes, and escape rate before introducing new gates.
- **Controlled experiments** — Enable one additional scanner or policy at a time and measure impact on duration, FPR, and developer satisfaction.
- **Synthetic vulnerability campaigns** — Periodically inject known-vulnerable components and verify detection + blocking behaviour (red-team style).
- **Historical trend analysis** — Track MTTD, MTTR, and severity distribution over sprints; visualise in dashboards (Grafana, custom security portal).
- **Developer experience surveys** — Qualitative feedback on noise, clarity of findings, and ease of remediation.
- **Production incident correlation** — Map security incidents back to whether the corresponding vulnerability was detectable by the existing workflow and why it was missed or accepted.

4.3 Balancing Security and Performance

Aggressive scanning on every commit can degrade developer velocity. Recommended practices include:

- Fast, incremental scans on Pull Requests (changed files / delta SCA) and deeper full scans on main-branch or nightly builds.
- Caching of dependency and image scan results to avoid redundant work.
- Parallel execution of independent security jobs.
- Severity-based gating — only Critical/High (or high-EPSS) findings block the pipeline; Medium/Low create tickets.
- Clear, actionable findings with remediation guidance and auto-generated fix PRs where possible (Dependabot, Snyk fix).
- Continuous tuning of rulesets to reduce false positives while preserving high true-positive rates for important classes of bugs.

5. Conclusion

Effective vulnerability management in a DevSecOps environment requires a closed loop of **detection** → **severity analysis** → **verification workflow** → **performance evaluation**. Detection must be multi-layered and continuous. Severity must incorporate real-world exploitability and business context, not only CVSS base scores. The verification workflow must be automated, auditable, and simulated regularly to prove its reliability. Finally, performance evaluation ensures that security controls improve risk posture without becoming a bottleneck. When these four elements operate together, organisations achieve both higher security assurance and sustainable delivery speed.

Key Takeaways

- Shift-left (SAST, SCA, secret, IaC) + shift-right (runtime, continuous re-scan) provides defence-in-depth.
- Use CVSS + EPSS + asset criticality to prioritise; enforce severity-based pipeline gates and SLAs.
- Model and regularly simulate the full security verification workflow with synthetic findings.
- Measure TPR, FPR, MTTD, MTTR, escape rate, and pipeline duration; optimise for both security and developer experience.